

INFORME INDIVIDUAL DE TRABAJO

Robert Altamar · Módulo de Lógica de Peticiones y Control de Conflictos · Proyecto g_agenda

Asignatura: Programación (IFCD0210 – Desenvolupament d'aplicacions amb tecnologia web)

Grupo: ROSIJO — **Fecha:** 22 de julio de 2026

Nota: *este documento es una base. Robert debe revisarlo y ajustarlo a lo que realmente implementó y sintió, porque el informe individual es subjetivo y personal.*

1. Mi parte en el proyecto (qué he hecho)

Mi responsabilidad fue el cerebro lógico de la aplicación: la etapa intermedia que decide, entre todas las peticiones de reserva, cuáles son válidas y cuáles no. Trabajé sobre la clase `Control` y me encargué de estos bloques:

- **Desglose de máscaras horarias:** programé el motor que rompe cadenas como `08-10-12-14-16-18-20-22` usando `split` en dos niveles (el guion bajo separa bloques de horario, el guion separa la hora de inicio de la de fin) para traducirlas a posiciones concretas dentro de la matriz.
- **Matriz bidimensional de ocupación:** gestioné una matriz de Strings de 24×32 (24 horas por los días del mes) donde cada celda representa una hora concreta de un día concreto en una sala.
- **Control de colisiones:** implementé las condiciones que comprueban si una celda ya está reservada antes de aceptar una nueva petición, dando prioridad total a los bloqueos de espacio con estado "Cerrado".
- **Persistencia de incidencias:** diseñé el volcado de las incidencias detectadas (reservas denegadas por solapamiento) hacia el fichero físico, junto con el resumen de horas asignadas.

2. Lo que pulimos y lo más complicado

El troceado de las máscaras dobles. Lo más complicado fue el parseo en dos niveles: una misma petición puede traer varios tramos horarios en la misma línea. Tuve que asegurarme de que cada tramo se convertía en el rango de filas correcto de la matriz sin dejarme ninguna hora ni pisar la siguiente.

El desacuerdo léxico con el frontend. En las primeras pruebas conjuntas, las celdas bloqueadas no se pintaban de gris. El motivo era que mi matriz inyectaba la palabra "Cerrado" mientras el frontend de Simon buscaba "Tancat". Fue un buen ejemplo de que en equipo hay que consensuar hasta las cadenas de texto de control: lo unificamos a "Cerrado" y el diseño empezó a renderizarse bien.

La prioridad de los bloqueos. Tuve que decidir un orden claro: un espacio marcado como cerrado gana siempre frente a cualquier reserva, aunque la reserva llegara antes. Traducir esa regla de negocio a condiciones lógicas estrictas me llevó varias pruebas.

3. Lo que me gustó y lo que me costó

Lo que me gustó. El control matricial. Ver cómo el sistema denegaba automáticamente una reserva que se solapaba con otra ya existente, y dejaba constancia de ello en un archivo de log, fue muy satisfactorio: es lógica de negocio real, del tipo que hay detrás de cualquier sistema de reservas.

Lo que me costó. Diseñar el control de colisiones de forma estricta y gestionar los flujos de salida para escribir en un fichero físico sin errores. Trabajar con la matriz y con la escritura de logs a la vez, sin perder la coordenada correcta, fue lo que más concentración me exigió.

4. Mi aporte al equipo

Fui la etapa intermedia del flujo: recibía la configuración y los idiomas que preparaba José Antonio, aplicaba las reglas del enunciado y entregaba a Simon una matriz ya resuelta, lista para dibujar. Sin esta capa lógica, el frontend no tendría datos coherentes que representar y no habría registro de incidencias.

5. Conclusión

Desarrollar el motor lógico me obligó a profundizar en el control de matrices y en el troceado de cadenas, que al principio se me hacían abstractos. El mayor aprendizaje ha sido diseñar un sistema de control de colisiones con prioridades claras y aprender a volcar datos formateados a un archivo físico. Me llevo, sobre todo, la importancia de consensuar los detalles con el equipo: un simple desacuerdo entre “Cerrado” y “Tancat” puede tumbar el renderizado entero.